CampusGuard: Design Plan & Work Split

COS 214 Practical 5 | Due 29 September 2026

1. Design Concept

Extra patterns chosen: **State** and **Composite**. Both fit the scenario naturally and neither overlaps with the required four (an Observer, for example, would compete with the Mediator).

State (incident lifecycle)

Each incident moves through Reported → Dispatched → Contained → Resolved, with Cancelled as an option. Each state object decides which operations are legal, so resolving an incident that was never dispatched is rejected cleanly. This also provides the required failure case without a large if/else chain.

Composite (campus areas)

Campus → Building → Floor → Room. Calling `lock()` on a building locks every room inside it, so building-access actions (lock, unlock, restrict) feel real instead of hardcoded.

Command (operator actions)

An `OperatorConsole` is the invoker and keeps a history. It executes `DispatchUnitCommand`, `LockAreaCommand` and `IssueAlertCommand`, and supports cancelling the last action via `undo()`. Receivers are real domain objects: incidents, response units and campus areas.

Mediator (coordination)

An `IncidentCoordinator` sits between the `SecurityTeam`, `MedicalTeam`, `FacilitiesTeam` and `AccessControlService` colleagues. When the medical team reports "on scene, need clear route", the coordinator tells access control to unlock the route, facilities to hold a lift, and security to manage the crowd. None of the teams know about each other.

Adapter (legacy system)

A `LegacyDoorController` exposes an awkward interface such as `sendCode(int zoneId, char opCode)`. CampusGuard wants an `AccessController` interface with `lock(Area&)` and `unlock(Area&)`. The adapter translates area objects into zone IDs and op codes. A legacy SMS/PA alert gateway would work equally well.

Facade (emergency workflows)

An `EmergencyWorkflows` class offers `evacuateBuilding(building)`, which in one call:

1. issues an alert,
2. unlocks all exits through the Composite and the Adapter,
3. dispatches security, and
4. updates the incident's state.

Each of those subsystems remains usable on its own.

2. Demo Stories

1. **Lab fire in the Engineering building.** The incident is reported, then the Facade runs an evacuation. This exercises the Adapter, Composite, State and Command in one flow, covering the "four or more patterns in one execution" requirement.
2. **Medical emergency in the library.** A dispatch command sends the medical team, and the team's update makes the Mediator coordinate the other teams. The operator then cancels a mistaken lockdown with undo, and an invalid state transition is attempted and rejected.

3. Work Split

	Person A	Person B	Person C
Code	Command (console, commands, undo) and State (incident lifecycle)	Mediator (coordinator, team colleagues) and Composite (campus areas)	Adapter (legacy door controller) and Facade, plus main and both scenario stories
Diagrams	Sequence diagram 1 (Command → Mediator flow) and the State diagram for the chosen pattern	Their section of the class diagram, then merging it into the final version	Sequence diagram 2 (Facade/Adapter evacuation)
Engineering	Valgrind evidence	GDB bug investigation	Makefile, Dockerfile, docker-compose, README
PDF	Justification for the two extra patterns	Ownership/destruction rationale	Assembling the final PDF and the collaboration reflection

Everyone writes their own GoF participant mapping and contribution statement.